

06 – Using Databases

Use your Databases in real life

Jérôme Landré – jerome.landre@ju.se

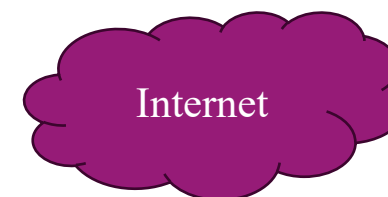
OUTLINE

1. Using Databases
2. Node.js
3. Node.js + Express
4. Node.js + Express + REST API
5. Node.js + Express + REST API + CRUD

1. Using Databases

- In real-case applications, you (generally) do not use directly a database alone
- You use an interface to interact with the database:
 - Web page (Internet)
 - Application (Computer or Phone)

Web pages



Application



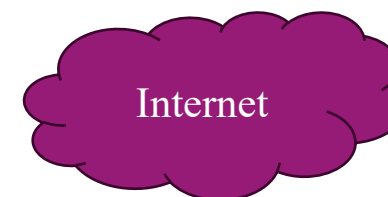
1. Using Databases

- In real-case applications, you (generally) do not use directly a database alone
- You use an interface to interact with the database:
 - Web page (Internet)
 - Application (Computer or Phone)

Web pages

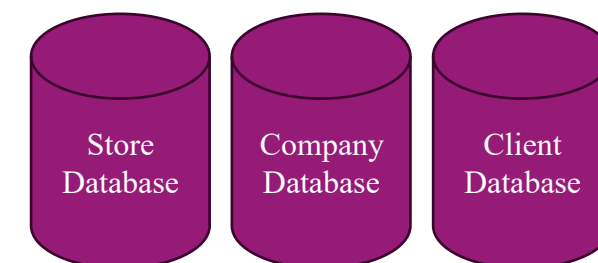


Node.js
Server



Database
Server

Application



2. Node.js

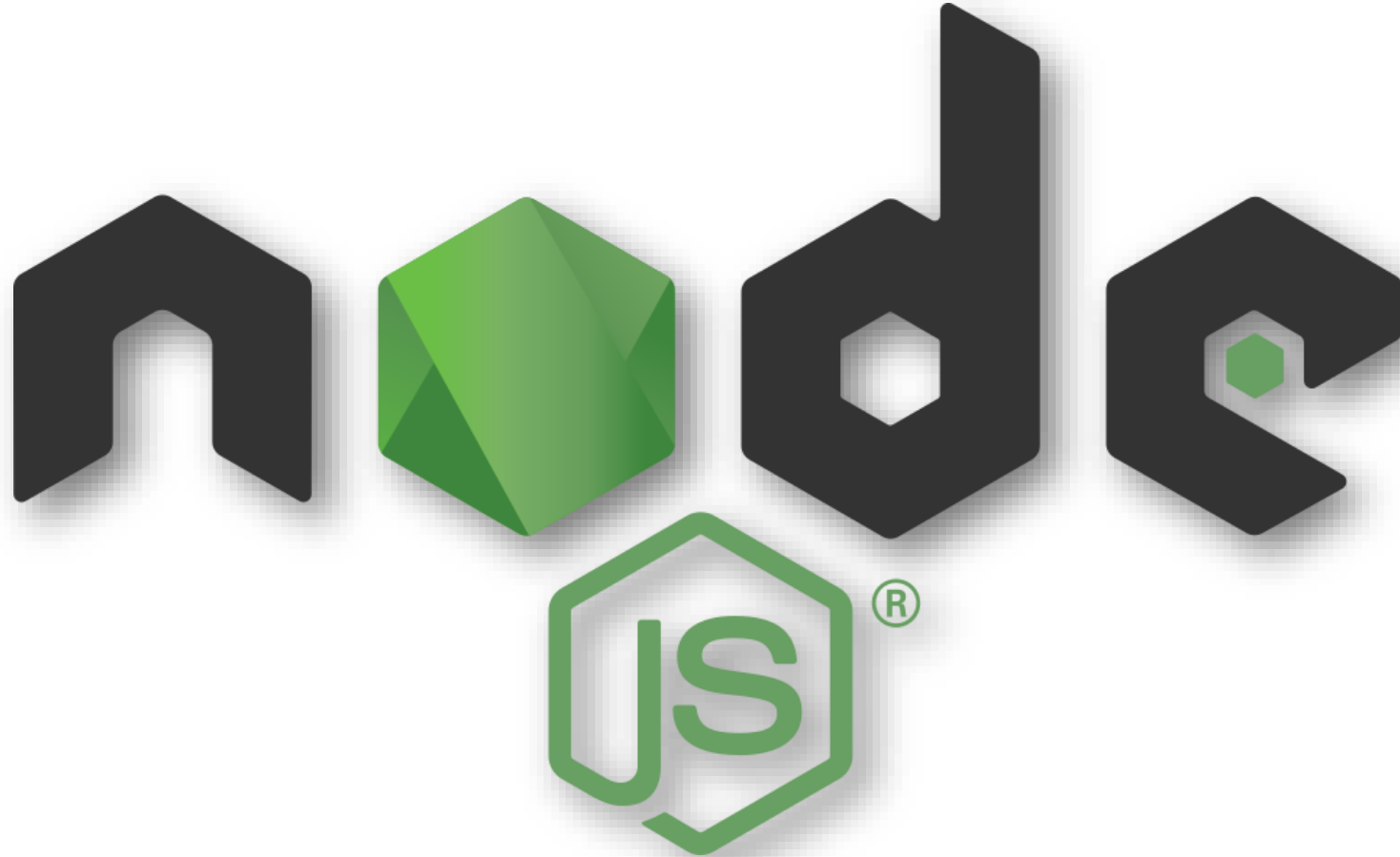
- **What is node.js in simple words?**

- **Node.js** is a free, open-source runtime environment that lets you run **JavaScript code outside a web browser**. In simple words:
- **It's JavaScript on the server:** Normally, JavaScript runs in your browser, but Node.js allows you to use it to build backend services, APIs, and even desktop applications.

- **Fast and efficient:** It's designed to handle many connections at once, making it great for real-time apps like chat or streaming.
- **Event-driven:** It uses events and callbacks to process tasks asynchronously, so it doesn't block other operations while waiting for something (like a database query) to finish.

Think of Node.js as a tool that lets JavaScript do more than just make websites interactive—it can power the entire backend of your application.

2. Node.js



Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Get Node.js®

Get security support
for EOL Node.js versions



Node.js is proudly supported by
the partners above and more.

Create an HTTP Server Write Tests Read and Hash a File Streams Pipeline Work with Threads

```
1 // server.mjs
2 import { createServer } from 'node:http';
3
4 const server = createServer((req, res) => {
5   res.writeHead(200, { 'Content-Type': 'text/plain' });
6   res.end('Hello World!\n');
7 });
8
9 // starts a simple http server locally on port 3000
10 server.listen(3000, '127.0.0.1', () => {
11   console.log('Listening on 127.0.0.1:3000');
12 });
13
14 // run with `node server.mjs`
```

JavaScript

[</> Copy to clipboard](#)

Learn more what Node.js is able to offer with our [Learning materials](#).

Download Node.js®

Get Node.js® v24.15.0 LTS for Windows using Docker with npm

Info Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:24-slim
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:24-slim
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.15.0".
12
13 # Verify npm version:
14 npm -v # Should print "11.12.1".
```

PowerShell

[</> Copy to clipboard](#)

Docker is a containerization platform. If you encounter any issues please visit [Docker's website](#)

Or get a prebuilt Node.js® for Windows running a x64 architecture.

[Windows Installer \(.msi\)](#)

[Standalone Binary \(.zip\)](#)

Read the [changelog](#) or [blog post](#) for this version.

Learn more about [Node.js releases](#), including the release schedule and LTS status.

Learn how to [verify](#) signed SHASUMS.

Looking for Node.js source? Download a signed [Node.js source](#) tarball.

Check out our [nightly](#) binaries or all [previous releases](#) or the [unofficial](#) binaries for other platforms.

Or get a prebuilt Node.js® for Windows running a x64 architecture.

 [Windows Installer \(.msi\)](#)

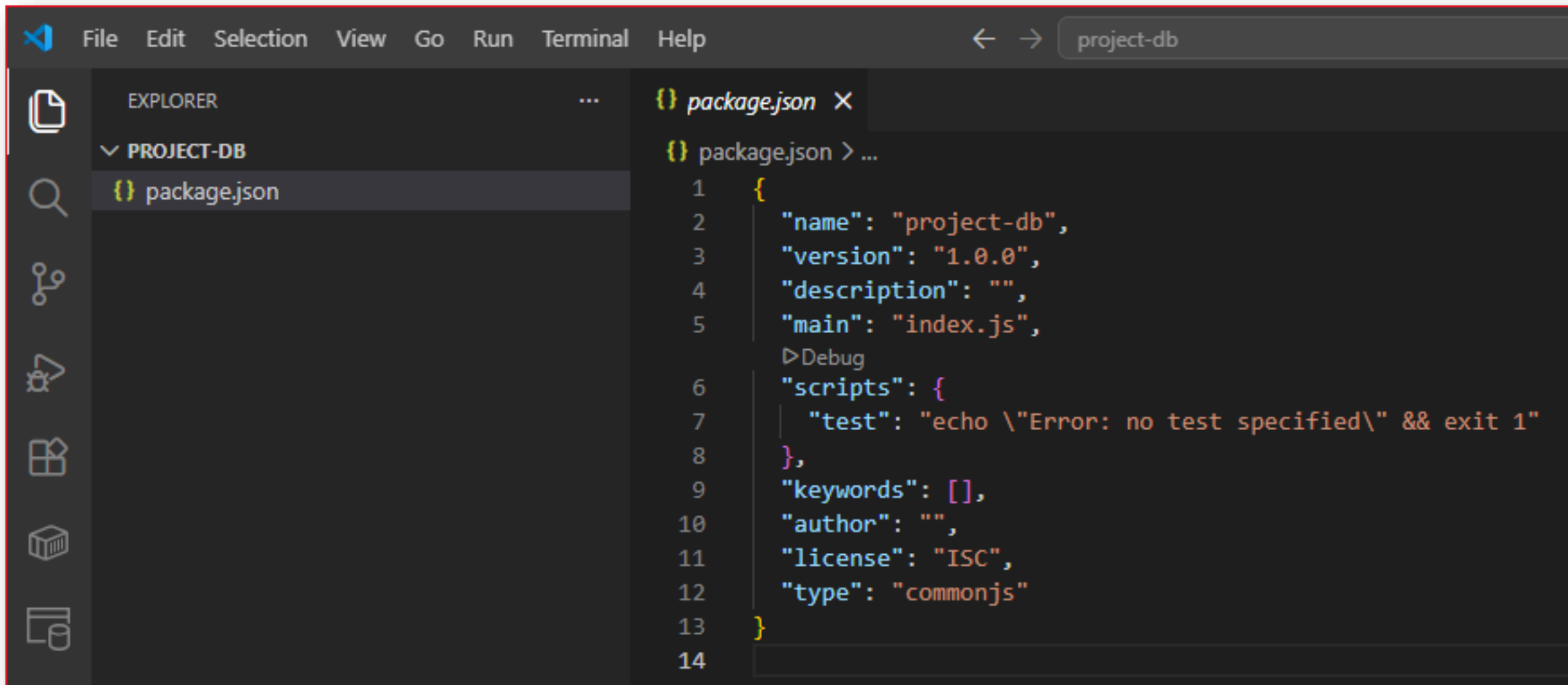
 [Standalone Binary \(.zip\)](#)

```
PS C:\temp> node --version
v24.11.1
PS C:\temp> node -v
v24.11.1
PS C:\temp> mkdir project-db
```

Directory: C:\temp

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	2026-04-27 07:41		project-db

```
PS C:\temp> cd .\project-db\  
PS C:\temp\project-db> npm init -y  
Wrote to C:\temp\project-db\package.json:  
  
{  
  "name": "project-db",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs"  
}  
  
PS C:\temp\project-db> |
```



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left displays a project named 'PROJECT-DB' with a file 'package.json' selected. The main editor area shows the content of 'package.json' with line numbers 1 through 14. The JSON configuration includes project name, version, description, main file, scripts, keywords, author, license, and type.

```
1  {
2    "name": "project-db",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "type": "commonjs"
13 }
14
```

```
1 CREATE DATABASE IF NOT EXISTS persondb;
2 USE persondb;
3 CREATE TABLE IF NOT EXISTS Person (
4     pid INT AUTO_INCREMENT PRIMARY KEY,
5     pfirstname VARCHAR(100), plastname VARCHAR(100),
6     pemail VARCHAR(255), pbirthdate DATE
7 );
8 INSERT INTO Person (pid, pfirstname, plastname, pbirthdate, pemail) VALUES
9 (1, 'John', 'Doe', '1990-01-01', 'john.doe@example.com'),(2, 'Jane', 'Smith', '1985-05-15', 'jane.smith@example.com'),
10 (3, 'Alice', 'Johnson', '1992-09-30', 'alice.johnson@example.com'),(4, 'Bob', 'Brown', '1988-12-20', 'bob.brown@example.com'),
11 (5, 'Charlie', 'Davis', '1995-03-10', 'charlie.davis@example.com'),(6, 'Emily', 'Wilson', '1991-07-25', 'emily.wilson@example.com'),
12 (7, 'David', 'Miller', '1987-11-05', 'david.miller@example.com'),(8, 'Sarah', 'Taylor', '1993-02-18', 'sarah.taylor@example.com'),
13 (9, 'Michael', 'Anderson', '1989-08-12', 'michael.anderson@example.com'),(10, 'Olivia', 'Thomas', '1994-04-22', 'olivia.thomas@example.com'),
14 (11, 'James', 'Jackson', '1986-10-30', 'james.jackson@example.com'),(12, 'Sophia', 'White', '1990-06-15', 'sophia.white@example.com'),
15 (13, 'William', 'Harris', '1988-01-05', 'william.harris@example.com'),(14, 'Isabella', 'Martin', '1992-11-20', 'isabella.martin@example.com'),
16 (15, 'Benjamin', 'Garcia', '1987-04-10', 'benjamin.garcia@example.com'),(16, 'Mia', 'Clark', '1993-09-05', 'mia.clark@example.com'),
17 (17, 'Jacob', 'Rodriguez', '1989-12-25', 'jacob.rodriguez@example.com'),(18, 'Charlotte', 'Lewis', '1991-03-15', 'charlotte.lewis@example.com'),
18 (19, 'Ethan', 'Lee', '1986-07-30', 'ethan.lee@example.com'),(20, 'Amelia', 'Walker', '1994-05-10', 'amelia.walker@example.com'),
19 (21, 'Alexander', 'Hall', '1988-09-25', 'alexander.hall@example.com'),(22, 'Evelyn', 'Allen', '1990-02-20', 'evelyn.allen@example.com');
20
```

```
MariaDB [persondb]> SELECT * FROM Person;
```

pid	pfirstname	plastname	pbirthdate	pemail
1	John	Doe	1990-01-01	john.doe@example.com
2	Jane	Smith	1985-05-15	jane.smith@example.com
3	Alice	Johnson	1992-09-30	alice.johnson@example.com
4	Bob	Brown	1988-12-20	bob.brown@example.com
5	Charlie	Davis	1995-03-10	charlie.davis@example.com
6	Emily	Wilson	1991-07-25	emily.wilson@example.com
7	David	Miller	1987-11-05	david.miller@example.com
8	Sarah	Taylor	1993-02-18	sarah.taylor@example.com
9	Michael	Anderson	1989-08-12	michael.anderson@example.com
10	Olivia	Thomas	1994-04-22	olivia.thomas@example.com
11	James	Jackson	1986-10-30	james.jackson@example.com
12	Sophia	White	1990-06-15	sophia.white@example.com
13	William	Harris	1988-01-05	william.harris@example.com
14	Isabella	Martin	1992-11-20	isabella.martin@example.com
15	Benjamin	Garcia	1987-04-10	benjamin.garcia@example.com
16	Mia	Clark	1993-09-05	mia.clark@example.com
17	Jacob	Rodriguez	1989-12-25	jacob.rodriguez@example.com
18	Charlotte	Lewis	1991-03-15	charlotte.lewis@example.com
19	Ethan	Lee	1986-07-30	ethan.lee@example.com
20	Amelia	Walker	1994-05-10	amelia.walker@example.com
21	Alexander	Hall	1988-09-25	alexander.hall@example.com
22	Evelyn	Allen	1990-02-20	evelyn.allen@example.com

```
22 rows in set (0.001 sec)
```

```
MariaDB [persondb]> █
```

The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows a project named 'PROJECT-DB' with files 'node_modules', 'package-lock.json', and 'package.json'. The 'package.json' file is open in the editor, showing the following content:

```
{} package.json X
{} package.json > ...
1  {
2    "name": "project-db",
3    "version": "1.0.0",
4    "description": "Using my MariaDB database with Node.js.",
5    "main": "server.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [
10     "database",
11     "nodejs",
12     "mariadb"
13   ],
14   "author": "JL/2026",
15   "license": "ISC",
16   "type": "commonjs",
17   "dependencies": {
18     "mysql2": "^3.22.3"
19   }
20 }
21
```

The Terminal panel at the bottom shows the output of the command `npm install mysql2` executed in a PowerShell window:

```
PS C:\TEMP\project-db> npm install mysql2

added 13 packages, and audited 14 packages in 3s

3 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\TEMP\project-db>
```

The status bar at the bottom indicates the current file is 'Ln 21, Col 1', with 'Spaces: 2', 'UTF-8', 'LF' encoding, and 'JSON' format. It also shows icons for Prettier and other extensions.

```
1  const http = require('http');
2  const fs    = require('fs');
3  const mysql = require('mysql2/promise');
4
5  const pool = mysql.createPool({
6    host: 'localhost', port: 3306,
7    user: 'root', password: 'pw1234', database: 'persondb'
8  });
9
10 http.createServer(async (req, res) => {
11   if (req.url === '/' || req.url === '/index.html') {
12     res.writeHead(200, { 'Content-Type': 'text/html' });
13     return res.end(fs.readFileSync('data.html'));
14   }
15   if (req.url === '/api/persons') {
16     const [rows] = await pool.query('SELECT * FROM Person');
17     res.writeHead(200, { 'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*' });
18     return res.end(JSON.stringify(rows));
19   }
20   res.writeHead(404); res.end('Not found');
21 }).listen(3000, () => console.log('Open http://localhost:3000'));
22
```

```

1  const http = require('http');
2  const fs   = require('fs');
3  const mysql = require('mysql2/promise');
4
5  const pool = mysql.createPool({
6    host: 'localhost', port: 3306,
7    user: 'root', password: 'pw1234', database: 'persondb'
8  });
9
10 http.createServer(async (req, res) => {
11   if (req.url === '/' || req.url === '/index.html') {
12     res.writeHead(200, { 'Content-Type': 'text/html' });
13     return res.end(fs.readFileSync('data.html'));
14   }
15   if (req.url === '/api/persons') {
16     const [rows] = await pool.query('SELECT * FROM Person');
17     res.writeHead(200, { 'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*' });
18     return res.end(JSON.stringify(rows));
19   }
20   res.writeHead(404); res.end('Not found');
21 }).listen(3000, () => console.log('Open http://localhost:3000'));
22

```

localhost:3000/api/persons

Pretty-print ☐

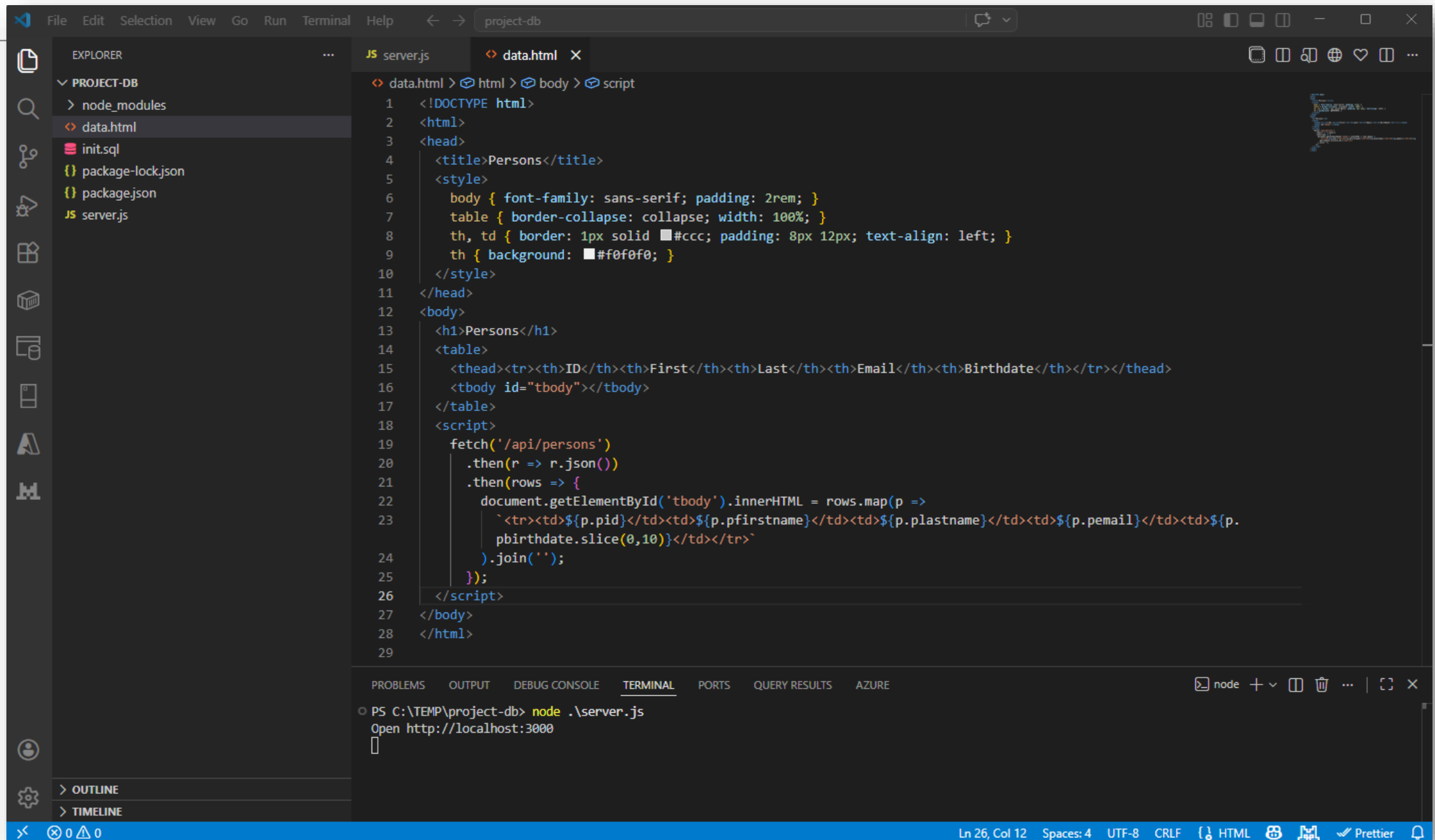
```

[{"pid":1,"pfirstname":"John","plastname":"Doe","pbirthdate":"1989-12-31T23:00:00.000Z","pemail":"john.doe@example.com"}, {"pid":2,"pfirstname":"Jane","plastname":"Smith","pbirthdate":"1985-05-14T22:00:00.000Z","pemail":"jane.smith@example.com"}, {"pid":3,"pfirstname":"Alice","plastname":"Johnson","pbirthdate":"1992-09-29T23:00:00.000Z","pemail":"alice.johnson@example.com"}, {"pid":4,"pfirstname":"Bob","plastname":"Brown","pbirthdate":"1988-12-19T23:00:00.000Z","pemail":"bob.brown@example.com"}, {"pid":5,"pfirstname":"Charlie","plastname":"Davis","pbirthdate":"1995-03-09T23:00:00.000Z","pemail":"charlie.davis@example.com"}, {"pid":6,"pfirstname":"Emily","plastname":"Wilson","pbirthdate":"1991-07-24T22:00:00.000Z","pemail":"emily.wilson@example.com"}, {"pid":7,"pfirstname":"David","plastname":"Miller","pbirthdate":"1987-11-04T23:00:00.000Z","pemail":"david.miller@example.com"}, {"pid":8,"pfirstname":"Sarah","plastname":"Taylor","pbirthdate":"1993-02-17T23:00:00.000Z","pemail":"sarah.taylor@example.com"}, {"pid":9,"pfirstname":"Michael","plastname":"Anderson","pbirthdate":"1989-08-11T22:00:00.000Z","pemail":"michael.anderson@example.com"}, {"pid":10,"pfirstname":"Olivia","plastname":"Thomas","pbirthdate":"1994-04-21T22:00:00.000Z","pemail":"olivia.thomas@example.com"}, {"pid":11,"pfirstname":"James","plastname":"Jackson","pbirthdate":"1986-10-29T23:00:00.000Z","pemail":"james.jackson@example.com"}, {"pid":12,"pfirstname":"Sophia","plastname":"White","pbirthdate":"1990-06-14T22:00:00.000Z","pemail":"sophia.white@example.com"}, {"pid":13,"pfirstname":"William","plastname":"Harris","pbirthdate":"1988-01-04T23:00:00.000Z","pemail":"william.harris@example.com"}, {"pid":14,"pfirstname":"Isabella","plastname":"Martin","pbirthdate":"1992-11-19T23:00:00.000Z","pemail":"isabella.martin@example.com"}, {"pid":15,"pfirstname":"Benjamin","plastname":"Garcia","pbirthdate":"1987-04-09T22:00:00.000Z","pemail":"benjamin.garcia@example.com"}, {"pid":16,"pfirstname":"Mia","plastname":"Clark","pbirthdate":"1993-09-04T22:00:00.000Z","pemail":"mia.clark@example.com"}, {"pid":17,"pfirstname":"Jacob","plastname":"Rodriguez","pbirthdate":"1989-12-24T23:00:00.000Z","pemail":"jacob.rodriquez@example.com"}, {"pid":18,"pfirstname":"Charlotte","plastname":"Lewis","pbirthdate":"1991-03-14T23:00:00.000Z","pemail":"charlotte.lewis@example.com"}, {"pid":19,"pfirstname":"Ethan","plastname":"Lee","pbirthdate":"1986-07-29T22:00:00.000Z","pemail":"ethan.lee@example.com"}, {"pid":20,"pfirstname":"Amelia","plastname":"Walker","pbirthdate":"1994-05-09T22:00:00.000Z","pemail":"amelia.walker@example.com"}, {"pid":21,"pfirstname":"Alexander","plastname":"Hall","pbirthdate":"1988-09-24T22:00:00.000Z","pemail":"alexander.hall@example.com"}, {"pid":22,"pfirstname":"Evelyn","plastname":"Allen","pbirthdate":"1990-02-19T23:00:00.000Z","pemail":"evelyn.allen@example.com"}]

```



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Persons</title>
5   <style>
6     body { font-family: sans-serif; padding: 2rem; }
7     table { border-collapse: collapse; width: 100%; }
8     th, td { border: 1px solid #ccc; padding: 8px 12px; text-align: left; }
9     th { background: #f0f0f0; }
10  </style>
11 </head>
12 <body>
13   <h1>Persons</h1>
14   <table>
15     <thead><tr><th>ID</th><th>First</th><th>Last</th><th>Email</th><th>Birthdate</th></tr></thead>
16     <tbody id="tbody"></tbody>
17   </table>
18   <script>
19     fetch('/api/persons')
20       .then(r => r.json())
21       .then(rows => {
22         document.getElementById('tbody').innerHTML = rows.map(p =>
23           `<tr><td>${p.pid}</td><td>${p.pfirstname}</td><td>${p.plastname}</td><td>${p.pemail}</td><td>${p.pbirthdate.slice(0,10)}</td></tr>`
24         ).join('');
25       });
26   </script>
27 </body>
28 </html>
29
```



Persons

localhost:3000

Persons

ID	First	Last	Email	Birthdate
1	John	Doe	john.doe@example.com	1989-12-31
2	Jane	Smith	jane.smith@example.com	1985-05-14
3	Alice	Johnson	alice.johnson@example.com	1992-09-29
4	Bob	Brown	bob.brown@example.com	1988-12-19
5	Charlie	Davis	charlie.davis@example.com	1995-03-09
6	Emily	Wilson	emily.wilson@example.com	1991-07-24
7	David	Miller	david.miller@example.com	1987-11-04
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17
9	Michael	Anderson	michael.anderson@example.com	1989-08-11
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21
11	James	Jackson	james.jackson@example.com	1986-10-29
12	Sophia	White	sophia.white@example.com	1990-06-14
13	William	Harris	william.harris@example.com	1988-01-04

3. Node.js + express

- What is Express in Node.js in simple words?

- **Express** in Node.js is like a **super simple, fast toolkit** for building web applications and APIs.
- In simple words:
 - It helps you **handle requests** (like when someone visits a webpage or submits a form).
 - It lets you define **routes** (e.g., "when someone goes to /home, show this page").

- It's **lightweight** and easy to use, so you can focus on writing the logic for your app instead of dealing with complex setup.
- Think of Express as the **framework** that makes it easy to create a server and manage how your Node.js app talks to the web.

server2-express.js

```
1  const express = require('express');
2  const fs = require('fs');
3  const mysql = require('mysql2/promise');
4
5  const app = express();
6  const pool = mysql.createPool({
7    host: 'localhost',
8    port: 3306,
9    user: 'root',
10   password: 'pw1234',
11   database: 'persondb'
12 });
13
14 // Serve static files (like data.html)
15 app.get('/', (req, res) => {
16   res.sendFile(__dirname + '/data.html');
17 });
```

```
18
19 // API endpoint to fetch persons
20 app.get('/api/persons', async (req, res) => {
21   try {
22     const [rows] = await pool.query('SELECT * FROM Person');
23     res.header('Access-Control-Allow-Origin', '*');
24     res.json(rows);
25   } catch (err) {
26     res.status(500).json({ error: err.message });
27   }
28 });
```

```
29
30 // Handle 404 for all other routes
31 app.use((req, res) => {
32   res.status(404).send('Not found');
33 });
34
35 // Start the server
36 app.listen(3000, () => {
37   console.log('Open http://localhost:3000');
38 });
39
40
```

```
JS server2-express.js > ...
1  const express = require('express');
2  const fs = require('fs');
3  const mysql = require('mysql2/promise');
4
5  const app = express();
6  const pool = mysql.createPool({
7    host: 'localhost',
8    port: 3306,
9    user: 'root',
10   password: 'pw1234',
11   database: 'persondb'
12 });
13
14 // Serve static files (like data.html)
15 app.get('/', (req, res) => {
16   res.sendFile(__dirname + '/data.html');
17 });
18
19 // API endpoint to fetch persons
20 app.get('/api/persons', async (req, res) => {
21   try {
22     const [rows] = await pool.query('SELECT * FROM Person');
23     res.header('Access-Control-Allow-Origin', '*');
24     res.json(rows);
25   } catch (err) {
26     res.status(500).json({ error: err.message });
27   }
28 });
29
30 // Handle 404 for all other routes
31 app.use((req, res) => {
32   res.status(404).send('Not found');
33 });
34
35 // Start the server
36 app.listen(3000, () => {
37   console.log('Open http://localhost:3000');
38 });
39
```



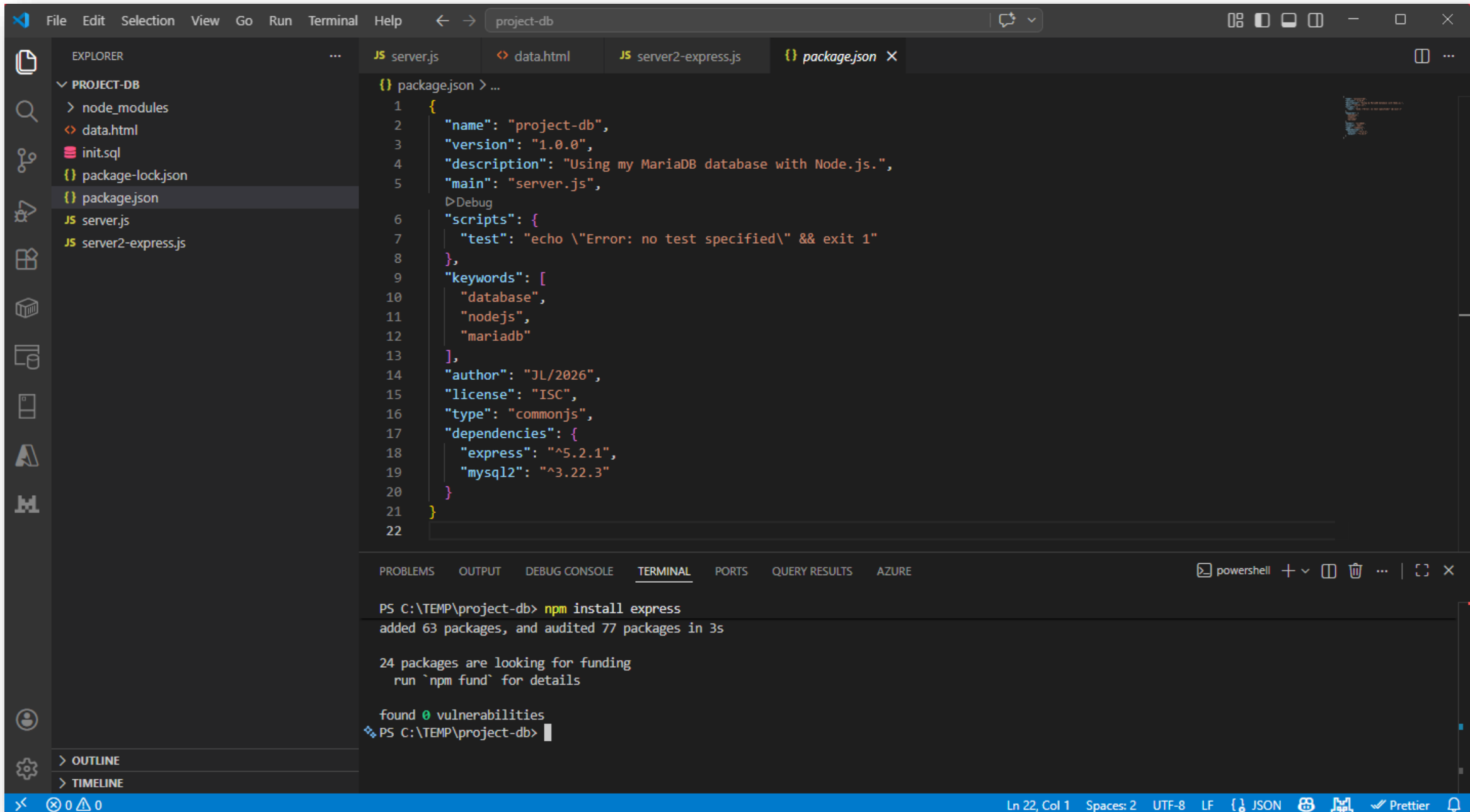
```
PS C:\TEMP\project-db> node .\server2-express.js
```

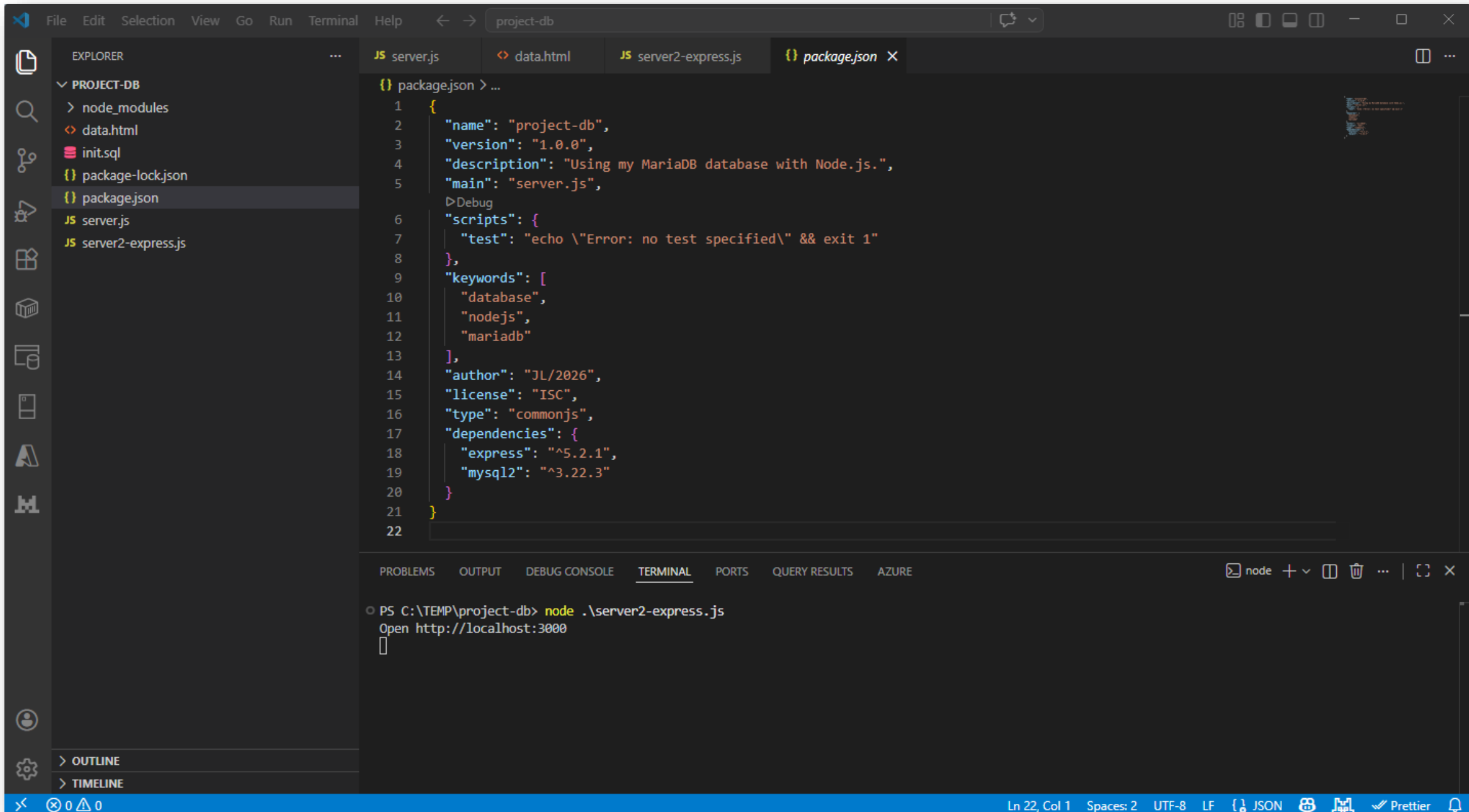
```
Require stack:
```

```
- C:\TEMP\project-db\server2-express.js
  at Module._resolveFilename (node:internal/modules/cjs/loader:1421:15)
  at defaultResolveImpl (node:internal/modules/cjs/loader:1059:19)
  at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1064:22)
  at Module._load (node:internal/modules/cjs/loader:1227:37)
  at TracingChannel.traceSync (node:diagnostics_channel:328:14)
  at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
  at Module.require (node:internal/modules/cjs/loader:1504:12)
  at require (node:internal/modules/helpers:152:16)
  at Object.<anonymous> (C:\TEMP\project-db\server2-express.js:1:17)
  at Module._compile (node:internal/modules/cjs/loader:1761:14) {
  code: 'MODULE_NOT_FOUND',
  requireStack: [ 'C:\\TEMP\\project-db\\server2-express.js' ]
}
```

```
Node.js v24.11.1
```

```
❖ PS C:\TEMP\project-db> █
```





Persons

localhost3000/api/persons

localhost3000/api/persons

Pretty-print

```
[{"pid":1,"pfirstname":"John","plastname":"Doe","pbirthdate":"1989-12-31T23:00:00.000Z","pemail":"john.doe@example.com"}, {"pid":2,"pfirstname":"Jane","plastname":"Smith","pbirthdate":"1985-05-14T22:00:00.000Z","pemail":"jane.smith@example.com"}, {"pid":3,"pfirstname":"Alice","plastname":"Johnson","pbirthdate":"1992-09-29T23:00:00.000Z","pemail":"alice.johnson@example.com"}, {"pid":4,"pfirstname":"Bob","plastname":"Brown","pbirthdate":"1988-12-19T23:00:00.000Z","pemail":"bob.brown@example.com"}, {"pid":5,"pfirstname":"Charlie","plastname":"Davis","pbirthdate":"1995-03-09T23:00:00.000Z","pemail":"charlie.davis@example.com"}, {"pid":6,"pfirstname":"Emily","plastname":"Wilson","pbirthdate":"1991-07-24T22:00:00.000Z","pemail":"emily.wilson@example.com"}, {"pid":7,"pfirstname":"David","plastname":"Miller","pbirthdate":"1987-11-04T23:00:00.000Z","pemail":"david.miller@example.com"}, {"pid":8,"pfirstname":"Sarah","plastname":"Taylor","pbirthdate":"1993-02-17T23:00:00.000Z","pemail":"sarah.taylor@example.com"}, {"pid":9,"pfirstname":"Michael","plastname":"Anderson","pbirthdate":"1989-08-11T22:00:00.000Z","pemail":"michael.anderson@example.com"}, {"pid":10,"pfirstname":"Olivia","plastname":"Thomas","pbirthdate":"1994-04-21T22:00:00.000Z","pemail":"olivia.thomas@example.com"}, {"pid":11,"pfirstname":"James","plastname":"Jackson","pbirthdate":"1986-10-29T23:00:00.000Z","pemail":"james.jackson@example.com"}, {"pid":12,"pfirstname":"Sophia","plastname":"White","pbirthdate":"1990-06-14T22:00:00.000Z","pemail":"sophia.white@example.com"}, {"pid":13,"pfirstname":"William","plastname":"Harris","pbirthdate":"1988-01-04T23:00:00.000Z","pemail":"william.harris@example.com"}, {"pid":14,"pfirstname":"Isabella","plastname":"Martin","pbirthdate":"1992-11-19T23:00:00.000Z","pemail":"isabella.martin@example.com"}, {"pid":15,"pfirstname":"Benjamin","plastname":"Garcia","pbirthdate":"1987-04-09T22:00:00.000Z","pemail":"benjamin.garcia@example.com"}, {"pid":16,"pfirstname":"Mia","plastname":"Clark","pbirthdate":"1993-09-04T22:00:00.000Z","pemail":"mia.clark@example.com"}, {"pid":17,"pfirstname":"Jacob","plastname":"Rodriguez","pbirthdate":"1989-12-24T23:00:00.000Z","pemail":"jacob.rodriguez@example.com"}, {"pid":18,"pfirstname":"Charlotte","plastname":"Lewis","pbirthdate":"1991-03-14T23:00:00.000Z","pemail":"charlotte.lewis@example.com"}, {"pid":19,"pfirstname":"Ethan","plastname":"Lee","pbirthdate":"1986-07-29T22:00:00.000Z","pemail":"ethan.lee@example.com"}, {"pid":20,"pfirstname":"Amelia","plastname":"Walker","pbirthdate":"1994-05-09T22:00:00.000Z","pemail":"amelia.walker@example.com"}, {"pid":21,"pfirstname":"Alexander","plastname":"Hall","pbirthdate":"1988-09-24T22:00:00.000Z","pemail":"alexander.hall@example.com"}, {"pid":22,"pfirstname":"Evelyn","plastname":"Allen","pbirthdate":"1990-02-19T23:00:00.000Z","pemail":"evelyn.allen@example.com"}]
```

Persons

localhost3000/api/persons

localhost3000

Persons

ID	First	Last	Email	Birthdate
1	John	Doe	john.doe@example.com	1989-12-31
2	Jane	Smith	jane.smith@example.com	1985-05-14
3	Alice	Johnson	alice.johnson@example.com	1992-09-29
4	Bob	Brown	bob.brown@example.com	1988-12-19
5	Charlie	Davis	charlie.davis@example.com	1995-03-09
6	Emily	Wilson	emily.wilson@example.com	1991-07-24
7	David	Miller	david.miller@example.com	1987-11-04
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17
9	Michael	Anderson	michael.anderson@example.com	1989-08-11
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21
11	James	Jackson	james.jackson@example.com	1986-10-29
12	Sophia	White	sophia.white@example.com	1990-06-14
13	William	Harris	william.harris@example.com	1988-01-04

4. Node.js + express + REST API

- Can you explain a REST API in simple words?
- A **REST API** is like a **waiter in a restaurant**:
 - **You (the client)** ask for something (e.g., "I want a pizza").
 - The **waiter (API)** takes your request to the **kitchen (server)**.
 - The kitchen prepares your order and sends it back to you via the waiter.
- In tech terms:
 - **REST API** is a set of rules for how software applications talk to each other over the internet.
 - It uses **HTTP** methods (like **GET, POST, PUT, DELETE**) to perform actions (**fetch data, create, update, or delete**).
 - It's stateless, meaning each request is independent and contains all the info needed to process it.
- Simple example:
 - You visit a website → Your browser sends a GET request to the server via the API → The server sends back the webpage data.
 - REST APIs power most modern apps (like social media, weather apps, etc.) by letting them fetch or send data easily!

JS server3-rest-api.js > ...

```
1  const express = require('express');
2  const mysql = require('mysql2/promise');
3
4  const app = express();
5  app.use(express.json()); // Middleware to parse JSON requests
6
7  // MySQL connection pool
8  const pool = mysql.createPool({
9    host: 'localhost',
10   port: 3306,
11   user: 'root',
12   password: 'pw1234',
13   database: 'persondb'
14 });
15
16 // Serve static files (like data.html)
17 app.get('/', (req, res) => {
18   res.sendFile(__dirname + '/data.html');
19 });
20
```

```
21 // --- GET: Fetch all persons ---
22 app.get('/api/persons', async (req, res) => {
23   try {
24     const [rows] = await pool.query('SELECT * FROM Person');
25     res.json(rows);
26   } catch (err) {
27     res.status(500).json({ error: err.message });
28   }
29 });
30
31 // --- GET: Fetch a single person by ID ---
32 app.get('/api/persons/:id', async (req, res) => {
33   try {
34     const [rows] = await pool.query('SELECT * FROM Person WHERE pid = ?', [req.params.id]);
35     if (rows.length === 0) {
36       return res.status(404).json({ error: 'Person not found' });
37     }
38     res.json(rows[0]);
39   } catch (err) {
40     res.status(500).json({ error: err.message });
41   }
42 });
43
```

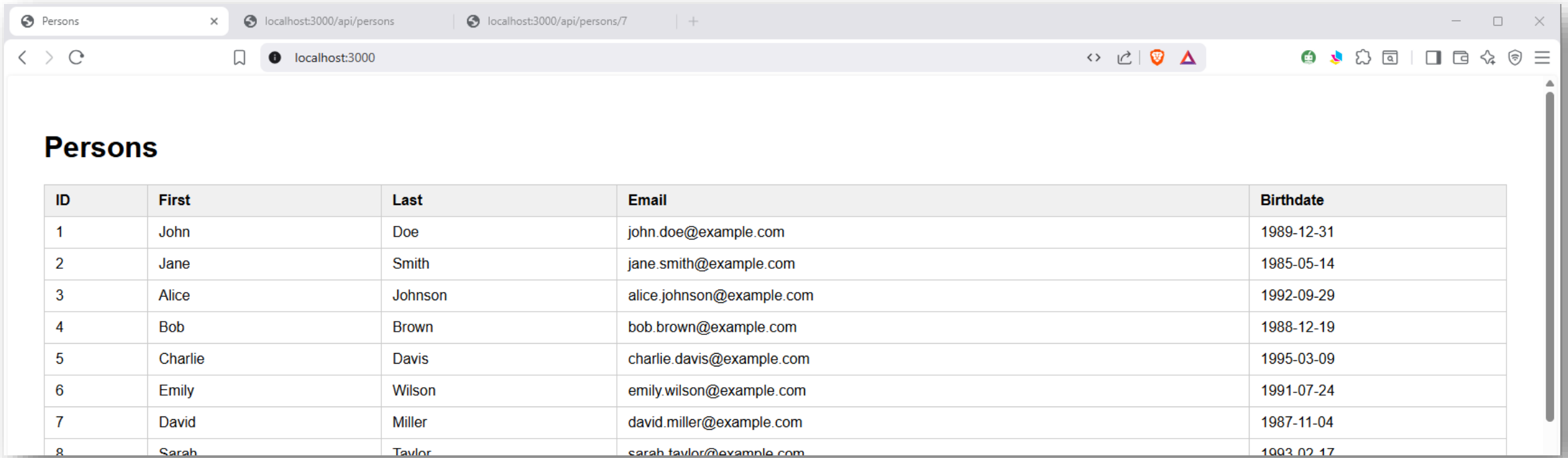
```
44 // --- POST: Create a new person ---
45 app.post('/api/persons', async (req, res) => {
46   try {
47     const { fname, lname, birthdate, email } = req.body;
48     if (!fname || !lname) {
49       return res.status(400).json({ error: 'First name and last name are required' });
50     }
51     const [result] = await pool.query(
52       'INSERT INTO Person (pfirstname, plastname, pbirthdate, pemail) VALUES (?, ?, ?, ?)',
53       [fname, lname, birthdate, email]
54     );
55     const [newPerson] = await pool.query('SELECT * FROM Person WHERE pid = ?', [result.insertId]);
56     res.status(201).json(newPerson[0]);
57   } catch (err) {
58     res.status(500).json({ error: err.message });
59   }
60 });
61
```



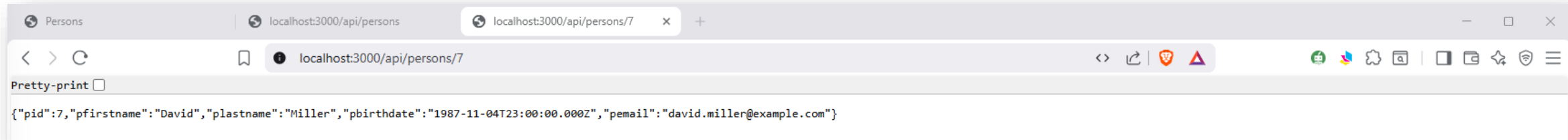
```
62 // --- PUT: Update a person by ID ---
63 app.put('/api/persons/:id', async (req, res) => {
64   try {
65     const { fname, lname, birthdate, email } = req.body;
66     if (!fname || !lname) {
67       return res.status(400).json({ error: 'First name and last name are required' });
68     }
69     const [result] = await pool.query(
70       'UPDATE Person SET pfirstname = ?, plastname = ?, pbirthdate = ?, pemail = ? WHERE pid = ?',
71       [fname, lname, birthdate, email, req.params.id]
72     );
73     if (result.affectedRows === 0) {
74       return res.status(404).json({ error: 'Person not found' });
75     }
76     const [updatedPerson] = await pool.query('SELECT * FROM Person WHERE pid = ?', [req.params.id]);
77     res.json(updatedPerson[0]);
78   } catch (err) {
79     res.status(500).json({ error: err.message });
80   }
81 });
82
```

```
83 // --- DELETE: Remove a person by ID ---
84 app.delete('/api/persons/:id', async (req, res) => {
85   try {
86     const [result] = await pool.query('DELETE FROM Person WHERE pid = ?', [req.params.id]);
87     if (result.affectedRows === 0) {
88       return res.status(404).json({ error: 'Person not found' });
89     }
90     res.status(204).end();
91   } catch (err) {
92     res.status(500).json({ error: err.message });
93   }
94 });
95
96 // Start the server
97 app.listen(3000, () => {
98   console.log('Server running on http://localhost:3000');
99 });
100
```

```
PS C:\TEMP\project-db> node .\server3-rest-api.js
Server running on http://localhost:3000
```



ID	First	Last	Email	Birthdate
1	John	Doe	john.doe@example.com	1989-12-31
2	Jane	Smith	jane.smith@example.com	1985-05-14
3	Alice	Johnson	alice.johnson@example.com	1992-09-29
4	Bob	Brown	bob.brown@example.com	1988-12-19
5	Charlie	Davis	charlie.davis@example.com	1995-03-09
6	Emily	Wilson	emily.wilson@example.com	1991-07-24
7	David	Miller	david.miller@example.com	1987-11-04
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17



```
curl -X DELETE http://localhost:3000/api/persons/1
```

```
PS C:\TEMP\project-db> Invoke-WebRequest -Uri "http://localhost:3000/api/persons/21" -Method DELETE
```

Security Warning: Script Execution Risk

Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.

RECOMMENDED ACTION:

Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "N"):y

```
StatusCode      : 204
StatusDescription : No Content
Content          : {}
RawContent       : HTTP/1.1 204 No Content
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Date: Mon, 27 Apr 2026 07:10:17 GMT
                  X-Powered-By: Express
```

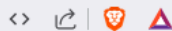
```
Headers         : {[Connection, keep-alive], [Keep-Alive, timeout=5],
                  [Date, Mon, 27 Apr 2026 07:10:17 GMT], [X-Powered-By, Express]}
```

```
RawContentLength : 0
```

```
PS C:\TEMP\project-db> 
```



localhost:3000



Persons

ID	First	Last	Email	Birthdate
1	John	Doe	john.doe@example.com	1989-12-31
2	Jane	Smith	jane.smith@example.com	1985-05-14
3	Alice	Johnson	alice.johnson@example.com	1992-09-29
4	Bob	Brown	bob.brown@example.com	1988-12-19
5	Charlie	Davis	charlie.davis@example.com	1995-03-09
6	Emily	Wilson	emily.wilson@example.com	1991-07-24
7	David	Miller	david.miller@example.com	1987-11-04
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17
9	Michael	Anderson	michael.anderson@example.com	1989-08-11
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21
11	James	Jackson	james.jackson@example.com	1986-10-29
12	Sophia	White	sophia.white@example.com	1990-06-14
13	William	Harris	william.harris@example.com	1988-01-04
14	Isabella	Martin	isabella.martin@example.com	1992-11-19
15	Benjamin	Garcia	benjamin.garcia@example.com	1987-04-09
16	Mia	Clark	mia.clark@example.com	1993-09-04
17	Jacob	Rodriguez	jacob.rodriguez@example.com	1989-12-24
18	Charlotte	Lewis	charlotte.lewis@example.com	1991-03-14
19	Ethan	Lee	ethan.lee@example.com	1986-07-29
20	Amelia	Walker	amelia.walker@example.com	1994-05-09
22	Evelyn	Allen	evelyn.allen@example.com	1990-02-19

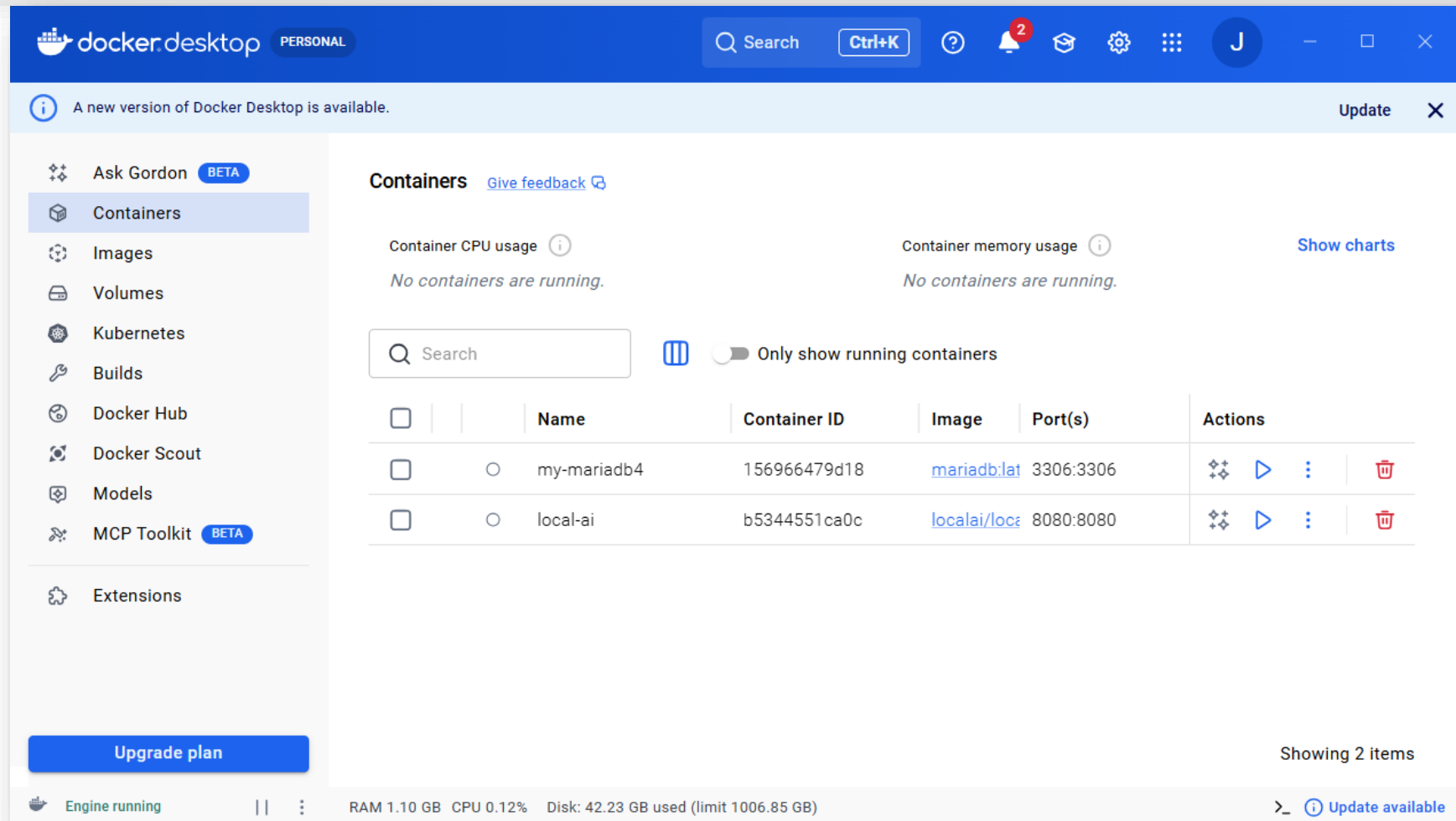
```

PS C:\TEMP\project-db> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
156966479d18   mariadb:latest "docker-entrypoint.s..." 6 days ago    Up 3 hours    0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp  my-mariadb4

PS C:\TEMP\project-db> docker stop my-mariadb4
my-mariadb4

PS C:\TEMP\project-db> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\TEMP\project-db>

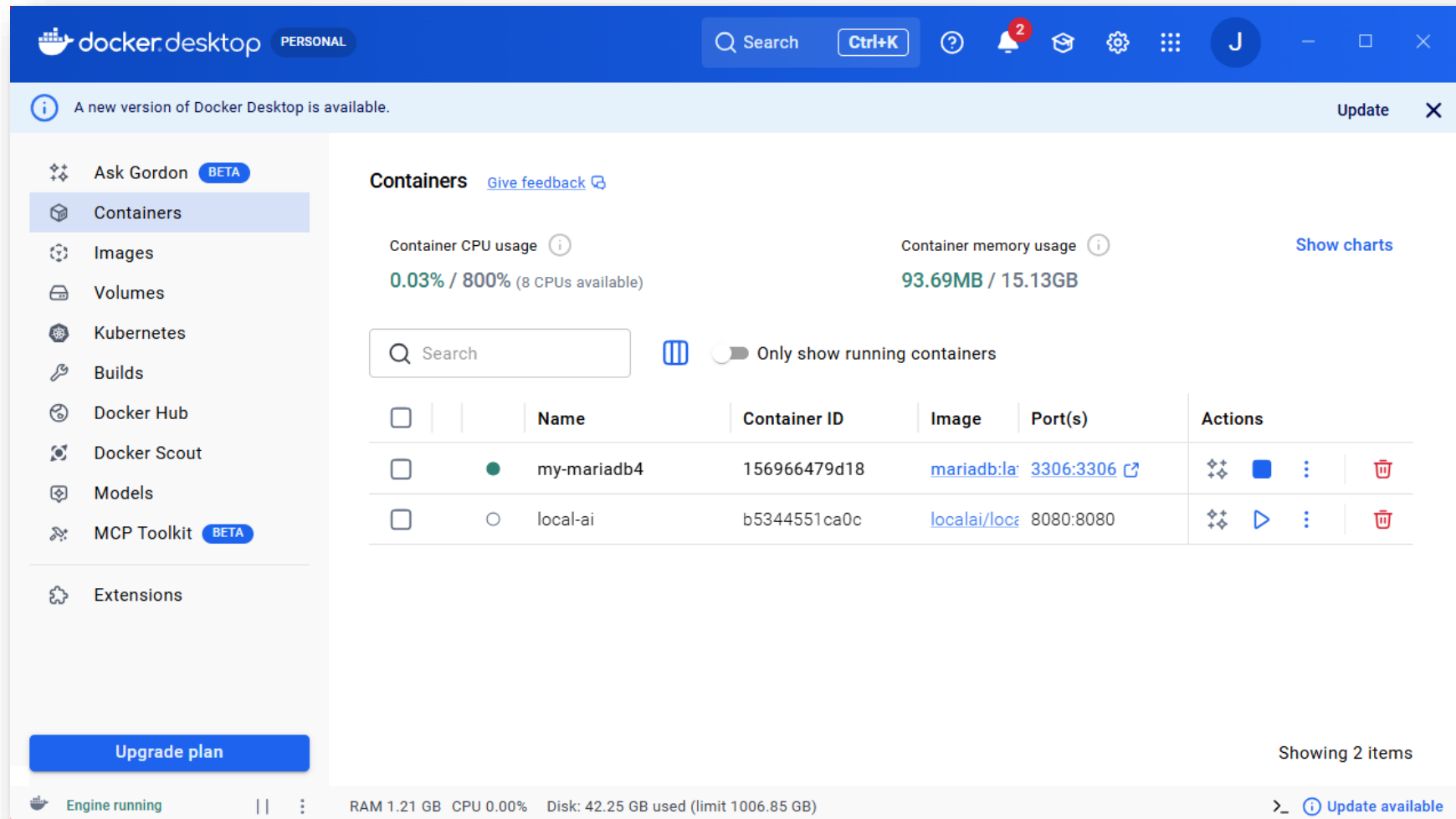
```



```
PS C:\TEMP\project-db> docker start my-mariadb4
my-mariadb4
PS C:\TEMP\project-db> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
156966479d18	mariadb:latest	"docker-entrypoint.s..."	6 days ago	Up 3 seconds	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp	my-mariadb4

```
PS C:\TEMP\project-db>
```



● PS C:\TEMP\project-db> docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
156966479d18	mariadb:latest	"docker-entrypoint.s..."	6 days ago	Up 57 seconds	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp	my-mariadb4

● PS C:\TEMP\project-db> docker images

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
code-nodeapp:latest	fa44ed7b938d	211MB	50MB	
docker/welcome-to-docker:latest	c4d56c24da4f	22.2MB	6.03MB	
files-api:latest	2bbea17a8010	232MB	54.5MB	
jer-img:latest	d9b98320fdb1	237MB	62.9MB	
localai/localai:latest	558efc7610e2	1.04GB	265MB	U
mariadb:11	78a5047d3ba3	465MB	111MB	
mariadb:latest	b3af559fc38e	467MB	111MB	U
my-db:latest	a6f15bbb6e0d	643MB	162MB	
nginx:latest	341bf0f3ce6c	240MB	65.8MB	
person-api-node:latest	1ff40595dfca	357MB	88.3MB	
welcome-to-docker:latest	ab3ea0bc1a90	483MB	131MB	

○ PS C:\TEMP\project-db>

5. Node.js + express + REST API + CRUD

- We can do better by processing the forms necessary to do **CREATE**, **UPDATE** and **DELETE**...

```
JS server3-rest-api.js > app.post('/api/persons') callback
1  const express = require('express');
2  const mysql = require('mysql2/promise');
3  const bodyParser = require('body-parser');
4
5  const app = express();
6  app.use(express.json()); // Middleware to parse JSON requests
7  app.use(bodyParser.urlencoded({ extended: true }));
8
9  // MySQL connection pool
10 const pool = mysql.createPool({
11   host: 'localhost',
12   port: 3306,
13   user: 'root',
14   password: 'pw1234',
15   database: 'persondb'
16 });
17
18 // Serve static files (like data.html)
19 app.get('/', (req, res) => {
20   res.sendFile(__dirname + '/data.html');
21 });
22
23 // --- GET: Fetch all persons ---
24 app.get('/api/persons', async (req, res) => {
25   console.log("----> SERVER: Fetching all persons...");
26   try {
27     const [rows] = await pool.query('SELECT * FROM Person');
28     res.json(rows);
29   } catch (err) {
30     res.status(500).json({ error: err.message });
31   }
32 });
```

```
34 // --- GET: Fetch a single person by ID ---
35 app.get('/api/persons/:id', async (req, res) => {
36   console.log("---> SERVER: Fetching person with ID: ", req.params.id);
37   try {
38     const [rows] = await pool.query('SELECT * FROM Person WHERE pid = ?', [req.params.id]);
39     if (rows.length === 0) {
40       return res.status(404).json({ error: 'Person not found' });
41     }
42     res.json(rows[0]);
43   } catch (err) {
44     res.status(500).json({ error: err.message });
45   }
46 });
47
48 // --- POST: Create a new person ---
49 app.post('/api/persons', async (req, res) => {
50   console.log("---> SERVER: Received POST request with body:", req.body);
51   try {
52     const { fn, ln, bdate, em } = req.body;
53     if (!fn || !ln) {
54       return res.status(400).json({ error: 'First name and last name are required' });
55     }
56     const [result] = await pool.query(
57       'INSERT INTO Person (pfirstname, plastname, pbirthdate, pemail) VALUES (?, ?, ?, ?)',
58       [fn, ln, bdate, em]
59     );
60     const [newPerson] = await pool.query('SELECT * FROM Person WHERE pid = ?', [result.insertId]);
61     res.status(201).json(newPerson[0]);
62   } catch (err) {
63     res.status(500).json({ error: err.message });
64   }
65 });
66
```

```
67 // --- PUT: Update a person by ID ---
68 app.put('/api/persons/:id', async (req, res) => {
69   console.log("---> SERVER: Received PUT request with body:", req.body);
70   try {
71     const { fname, lname, birthdate, email } = req.body;
72     if (!fname || !lname) {
73       return res.status(400).json({ error: 'First name and last name are required' });
74     }
75     const [result] = await pool.query(
76       'UPDATE Person SET pfirstname = ?, plastname = ?, pbirthdate = ?, pemail = ? WHERE pid = ?',
77       [fname, lname, birthdate, email, req.params.id]
78     );
79     if (result.affectedRows === 0) {
80       return res.status(404).json({ error: 'Person not found' });
81     }
82     const [updatedPerson] = await pool.query('SELECT * FROM Person WHERE pid = ?', [req.params.id]);
83     res.json(updatedPerson[0]);
84   } catch (err) {
85     res.status(500).json({ error: err.message });
86   }
87 });
88
89 // --- DELETE: Remove a person by ID ---
90 app.delete('/api/persons/:id', async (req, res) => {
91   console.log("---> SERVER: Received DELETE request for person with ID: ", req.params.id);
92   try {
93     const [result] = await pool.query('DELETE FROM Person WHERE pid = ?', [req.params.id]);
94     if (result.affectedRows === 0) {
95       return res.status(404).json({ error: 'Person not found' });
96     }
97     res.status(204).end();
98   } catch (err) {
99     res.status(500).json({ error: err.message });
100   }
101 });
102
103 // Start the server
104 app.listen(3000, () => {
105   console.log('Server running on http://localhost:3000');
106 });
```

data.html > html > body > script > newPerson

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Persons</title>
5    <style>
6      body { font-family: sans-serif; padding: 2rem; }
7      table { border-collapse: collapse; width: 100%; }
8      th, td { border: 1px solid #ccc; padding: 8px 12px; text-align: left; }
9      th { background: #f0f0f0; }
10   </style>
11 </head>
12 <body>
13   <h1>Persons</h1>
14   <form id="personForm" method="post" onsubmit="event.preventDefault(); newPerson();">
15     <input type="text" name="fn" id="fname" placeholder="First Name">
16     <input type="text" name="ln" id="lname" placeholder="Last Name">
17     <input type="email" name="em" id="email" placeholder="Email">
18     <input type="date" name="bdate" id="birthdate" placeholder="Birthdate">
19     <button type="submit">Add Person</button>
20   </form>
21   <p> </p>
22   <table>
23     <thead><tr><th>ID</th><th>First</th><th>Last</th><th>Email</th><th>Birthdate</th><th>Actions</th></tr></thead>
24     <tbody id="tbody"></tbody>
25   </table>
26   <script>
27     fetch('/api/persons')
28       .then(r => r.json())
29       .then(rows => {
30         console.log("----> Fetching persons...");
31         document.getElementById('tbody').innerHTML = rows.map(p =>
32           `<tr><td>${p.pid}</td><td>${p.pfirstname}</td><td>${p.plastname}</td><td>${p.pemail}</td><td>${p.pbirthdate.slice(0,10)}</td><td><button onclick="deletePerson(${p.
33             pid})">Delete</button><button onclick="updatePerson(${p.pid})">Update</button></td></tr>`
34           ).join('');
35       });

```

```
35 function deletePerson(pid) {
36   fetch(`/api/persons/${pid}`, { method: 'DELETE' })
37   .then(() => { console.log("----> Deleting person..."); location.reload(); });
38 }
39 function updatePerson(pid) {
40   console.log("----> Updating person with ID: ", pid);
41   const fname = document.getElementById('fname').value;
42   const lname = document.getElementById('lname').value;
43   const email = document.getElementById('email').value;
44   const birthdate = document.getElementById('birthdate').value;
45   if (fname && lname && email && birthdate) {
46     fetch(`/api/persons/${pid}`, {
47       method: 'PUT',
48       headers: { 'Content-Type': 'application/json' },
49       body: JSON.stringify({ pfirstname: fname, plastname: lname, pemail: email, pbirthdate: birthdate })
50     }).then(() => { console.log("----> Updating person..."); location.reload(); });
51   }
52 }
53 function newPerson() {
54   console.log("----> Creating new person...");
55   const fname = document.getElementById('fname').value;
56   const lname = document.getElementById('lname').value;
57   const email = document.getElementById('email').value;
58   const birthdate = document.getElementById('birthdate').value;
59   if (fname && lname && email && birthdate) {
60     fetch('/api/persons', {
61       method: 'POST',
62       headers: { 'Content-Type': 'application/json' },
63       body: JSON.stringify({ fn: fname, ln: lname, em: email, bdate: birthdate })
64     }).then(() => { console.log("----> Creating new person..."); location.reload(); });
65   }
66 }
67 </script>
68 </body>
69 </html>
70
```

READ (CRUD) /// SELECT ... FROM ... (SQL) /// GET (http)

Persons

First Name
 Last Name
 Email
 mm/dd/yyyy

ID	First	Last	Email	Birthdate	Actions
1	John	Doe	john.doe@example.com	1989-12-31	Delete Update
3	Alice	Johnson	alice.johnson@example.com	1992-09-29	Delete Update
4	Bob	Brown	bob.brown@example.com	1988-12-19	Delete Update
5	Charlie	Davis	charlie.davis@example.com	1995-03-09	Delete Update
7	David	Miller	david.miller@example.com	1987-11-04	Delete Update
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17	Delete Update
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21	Delete Update
11	James	Jackson	james.jackson@example.com	1986-10-29	Delete Update
12	Sophia	White	sophia.white@example.com	1990-06-14	Delete Update
13	William	Harris	william.harris@example.com	1988-01-04	Delete Update
14	Isabella	Martin	isabella.martin@example.com	1992-11-19	Delete Update
15	Benjamin	Garcia	benjamin.garcia@example.com	1987-04-09	Delete Update
16	Mia	Clark	mia.clark@example.com	1993-09-04	Delete Update
17	Jacob	Rodriguez	jacob.rodriguez@example.com	1980-12-24	Delete Update

Persons

First Name Last Name Email mm/dd/yyyy Add Person

ID	First	Last	Email	Birthdate	Actions
1	John	Doe	john.doe@example.com	1989-12-31	Delete Update
3	Alice	Johnson	alice.johnson@example.com	1992-09-29	Delete Update
4	Bob	Brown	bob.brown@example.com	1988-12-19	Delete Update
5	Charlie	Davis	charlie.davis@example.com	1995-03-09	Delete Update
7	David	Miller	david.miller@example.com	1987-11-04	Delete Update
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17	Delete Update
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21	Delete Update

DELETE (CRUD) /// DELETE FROM ... (SQL) /// DELETE (http)

Persons

First Name Last Name Email mm/dd/yyyy Add Person

ID	First	Last	Email	Birthdate	Actions
1	John	Doe	john.doe@example.com	1989-12-31	Delete Update
3	Alice	Johnson	alice.johnson@example.com	1992-09-29	Delete Update
4	Bob	Brown	bob.brown@example.com	1988-12-19	Delete Update
7	David	Miller	david.miller@example.com	1987-11-04	Delete Update
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17	Delete Update
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21	Delete Update
11	James	Jackson	james.jackson@example.com	1986-10-29	Delete Update

CREATE (CRUD) /// INSERT INTO ... (SQL) /// POST (http)

Persons

Jérôme Landré jl@ju.se 10/03/1972 Add Person

ID	First	Last	Email	Birthdate	Actions
1	John	Doe	john.doe@example.com	1989-12-31	Delete Update
3	Alice	Johnson	alice.johnson@example.com	1992-09-29	Delete Update
4	Bob	Brown	bob.brown@example.com	1988-12-19	Delete Update
7	David	Miller	david.miller@example.com	1987-11-04	Delete Update
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17	Delete Update
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21	Delete Update
11	James	Jackson	james.jackson@example.com	1986-10-29	Delete Update
12	Sophia	White	sophia.white@example.com	1990-06-14	Delete Update
13	William	Harris	william.harris@example.com	1988-01-04	Delete Update
14	Isabella	Martin	isabella.martin@example.com	1992-11-19	Delete Update
15	Benjamin	Garcia	benjamin.garcia@example.com	1987-04-09	Delete Update
16	Mia	Clark	mia.clark@example.com	1993-09-04	Delete Update
17	Jacob	Rodriguez	jacob.rodriguez@example.com	1989-12-24	Delete Update
18	Charlotte	Lewis	charlotte.lewis@example.com	1991-03-14	Delete Update
19	Ethan	Lee	ethan.lee@example.com	1986-07-29	Delete Update
20	Amelia	Walker	amelia.walker@example.com	1994-05-09	Delete Update
22	Evelyn	Allen	evelyn.allen@example.com	1990-02-19	Delete Update

CREATE (CRUD) /// INSERT INTO ... (SQL) /// POST (http)

Persons

ID	First	Last	Email	Birthdate	Actions
1	John	Doe	john.doe@example.com	1989-12-31	Delete Update
3	Alice	Johnson	alice.johnson@example.com	1992-09-29	Delete Update
4	Bob	Brown	bob.brown@example.com	1988-12-19	Delete Update
7	David	Miller	david.miller@example.com	1987-11-04	Delete Update
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17	Delete Update
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21	Delete Update
11	James	Jackson	james.jackson@example.com	1986-10-29	Delete Update
12	Sophia	White	sophia.white@example.com	1990-06-14	Delete Update
13	William	Harris	william.harris@example.com	1988-01-04	Delete Update
14	Isabella	Martin	isabella.martin@example.com	1992-11-19	Delete Update
15	Benjamin	Garcia	benjamin.garcia@example.com	1987-04-09	Delete Update
16	Mia	Clark	mia.clark@example.com	1993-09-04	Delete Update
17	Jacob	Rodriguez	jacob.rodriguez@example.com	1989-12-24	Delete Update
18	Charlotte	Lewis	charlotte.lewis@example.com	1991-03-14	Delete Update
19	Ethan	Lee	ethan.lee@example.com	1986-07-29	Delete Update
20	Amelia	Walker	amelia.walker@example.com	1994-05-09	Delete Update
22	Evelyn	Allen	evelyn.allen@example.com	1990-02-19	Delete Update
27	Jérôme	Landré	jl@ju.se	1972-10-02	Delete Update

UPDATE (CRUD) /// UPDATE ... (SQL) /// PUT (http)

Persons

Jeremy TheLandre jtl@google.com 07 / 27 / 2001 Add Person

ID	First	Last	Email	Birthdate	Actions
1	John	Doe	john.doe@example.com	1989-12-31	Delete Update
3	Alice	Johnson	alice.johnson@example.com	1992-09-29	Delete Update
4	Bob	Brown	bob.brown@example.com	1988-12-19	Delete Update
7	David	Miller	david.miller@example.com	1987-11-04	Delete Update
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17	Delete Update
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21	Delete Update
11	James	Jackson	james.jackson@example.com	1986-10-29	Delete Update
12	Sophia	White	sophia.white@example.com	1990-06-14	Delete Update
13	William	Harris	william.harris@example.com	1988-01-04	Delete Update
14	Isabella	Martin	isabella.martin@example.com	1992-11-19	Delete Update
15	Benjamin	Garcia	benjamin.garcia@example.com	1987-04-09	Delete Update
16	Mia	Clark	mia.clark@example.com	1993-09-04	Delete Update
17	Jacob	Rodriguez	jacob.rodriguez@example.com	1989-12-24	Delete Update
18	Charlotte	Lewis	charlotte.lewis@example.com	1991-03-14	Delete Update
19	Ethan	Lee	ethan.lee@example.com	1986-07-29	Delete Update
20	Amelia	Walker	amelia.walker@example.com	1994-05-09	Delete Update
22	Evelyn	Allen	evelyn.allen@example.com	1990-02-19	Delete Update
28	Jérôme	Landré	jl@ju.se	1972-10-02	Delete Update

UPDATE (CRUD) /// UPDATE ... (SQL) /// PUT (http)

Persons

ID	First	Last	Email	Birthdate	Actions
1	John	Doe	john.doe@example.com	1989-12-31	Delete Update
3	Alice	Johnson	alice.johnson@example.com	1992-09-29	Delete Update
4	Bob	Brown	bob.brown@example.com	1988-12-19	Delete Update
7	David	Miller	david.miller@example.com	1987-11-04	Delete Update
8	Sarah	Taylor	sarah.taylor@example.com	1993-02-17	Delete Update
10	Olivia	Thomas	olivia.thomas@example.com	1994-04-21	Delete Update
11	James	Jackson	james.jackson@example.com	1986-10-29	Delete Update
12	Sophia	White	sophia.white@example.com	1990-06-14	Delete Update
13	William	Harris	william.harris@example.com	1988-01-04	Delete Update
14	Isabella	Martin	isabella.martin@example.com	1992-11-19	Delete Update
15	Benjamin	Garcia	benjamin.garcia@example.com	1987-04-09	Delete Update
16	Mia	Clark	mia.clark@example.com	1993-09-04	Delete Update
17	Jacob	Rodriguez	jacob.rodriguez@example.com	1989-12-24	Delete Update
18	Charlotte	Lewis	charlotte.lewis@example.com	1991-03-14	Delete Update
19	Ethan	Lee	ethan.lee@example.com	1986-07-29	Delete Update
20	Amelia	Walker	amelia.walker@example.com	1994-05-09	Delete Update
22	Evelyn	Allen	evelyn.allen@example.com	1990-02-19	Delete Update
28	Jeremy	TheLandre	jtl@google.com	2001-07-26	Delete Update

**CONGRATULATIONS, GRATTIS, FÉLICITATIONS,
YOU DID THE CRUD ON YOUR DATABASE!!!**

ANY QUESTIONS?

BE POSITIVE, B+!